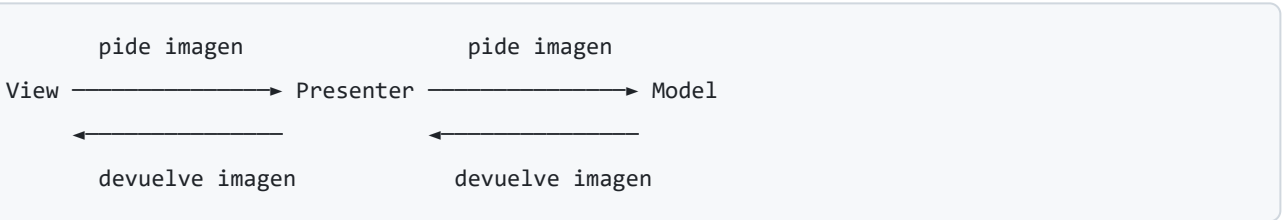
1. Flujo general

Las tres piezas

El programa se divide en tres partes con trabajos distintos:

Pieza	Su trabajo	Clases
Modelo	Calcular el fuego y generar la imagen	`Model`, `Fire`, `Palette`, `Render`
Vista	Enseñar la imagen en una ventana	`View`, `Viewer`
Presenter	Conectar las dos anteriores	`Presenter`

La regla importante: **el modelo y la vista no se conocen**. El modelo no sabe que existe una ventana, y la ventana no sabe cómo se calcula el fuego. Cuando la vista quiere la imagen, se la pide al Presenter, y este se la pide al modelo.



Así, si mañana cambias la ventana por otra cosa, el modelo no se toca.

Qué pasa al ejecutar, paso a paso

1. Arranca `Main`. Dentro de `invokeLater` crea las tres piezas:

```
IModel model = new Model();
IView view = new View();
IPresenter presenter = new Presenter(model, view);
presenter.start();
```

2. **`new Model()`** (`Model.java:14`) crea `Fire` (320 × 200 puntos, todos a 0) y `Render` (con la paleta del fuego).

3. **`new View()`** (`View.java:18`) crea el lienzo `Viewer` y la ventana `JFrame`, pero **todavía no la muestra**.

4. **`presenter.start()`** (`Presenter.java:21`) hace tres cosas, en este orden:

1. `view.setPresenter(this)`: la vista conoce al Presenter para poder pedirle la imagen.

2. `model.start()`: se arranca el hilo que hace avanzar el fuego.
3. `view.start()`: se abre la ventana y se arranca el hilo que la refresca.

5. Desde ese momento hay dos hilos trabajando a la vez:

Hilo	Dónde	Cada cuánto	Qué hace
`Model-Fire`	`Model.run`	33 ms (~30 veces/s)	Hace avanzar el fuego un paso
`fire-view`	`View.run`	16 ms (~60 veces/s)	Pide la imagen y la dibuja

La vista refresca más rápido de lo que el fuego cambia. No pasa nada: simplemente a veces repinta la misma imagen.

Un fotograma completo, de principio a fin

Esto es lo que pasa cada vez que se ve un cambio en pantalla:

```
Hilo Model-Fire                          Hilo fire-view
_______________                          _____________

update()                                 presenter.getImage()
 ├─ sparksGeneration()  enciende la base  └─ model.getImage()
 ├─ coolPointsGeneration() apaga parte       └─ render.getImage()
 └─ updateFlames()      sube el calor            ├─ recorre cada punto
sleep(33 ms)                                     ├─ temperatura → color (Palette)
                                                 └─ pinta cada píxel
                                         viewer.setImage(imagen)
                                          └─ repaint() → paint() dibuja
                                         sleep(16 ms)
```

Las temperaturas viven en `Fire`. Los colores se calculan en `Palette`. La imagen se rellena en `Render`.

Por qué hay interfaces (`IModel`, `IView`, `IPresenter`)

El `Presenter` guarda un `IModel` y un `IView`, no un `Model` y un `View`. Solo sabe los métodos que prometen los contratos:

- `IModel`: `update()`, `start()`, `getImage()`
- `IView`: `setPresenter(...)`, `start()`
- `IPresenter`: `start()`, `getImage()`

Así cada pieza depende del contrato y no de cómo está hecha la otra.